Flipped Classroom - Arrays (Data Structures)

College: John Cox Memorial CSI Institute of Technology

Faculty: Dr. B.S. Charulatha

Subject: Data Structure

Topic : Arrays

Date:

Aim

To understand the concept, structure, and implementation of Arrays in Data Structures and demonstrate their use in storing and manipulating data efficiently.

Theory

An Array is a collection of elements of the same data type stored in contiguous memory locations. Each element in an array can be accessed using an index. Arrays are useful for storing large sets of similar data efficiently and form the basis of more complex data structures like stacks and queues.

Features of Arrays

- Stores multiple values under a single variable name.
- Elements are of the same data type.
- Stored in contiguous memory locations.
- Can be accessed using index numbers.
- Fixed in size.

Types of Arrays

1. One-Dimensional Array – Elements are arranged in a single row.
   Example: int marks[5] = {90, 85, 70, 88, 95};

2. Two-Dimensional Array – Elements are arranged in rows and columns.
   Example:

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
```

```
};
```

3. Multi-Dimensional Array – Arrays with more than two dimensions, used in advanced applications.

Algorithm (Sum of Array Elements)

1. Start
2. Declare an array and variables for sum and loop counter.
3. Initialize the array with elements.
4. Traverse the array using a loop.
5. Add each element to the sum.
6. Display the total sum.
7. Stop.

Example Program (in C)

```
#include <stdio.h>
int main() {
   int arr[5] = {10, 20, 30, 40, 50};
   int sum = 0, i;
   for(i = 0; i < 5; i++) {
      sum += arr[i];
   }
   printf("Sum = %d", sum);
   return 0;
}
```

Output

Sum = 150

Advantages

- Easy to access elements using index.
- Reduces code complexity.
- Efficient for looping and data manipulation.

Disadvantages

- Fixed size – cannot resize once declared.
- Insertion and deletion operations are costly.
- May waste memory if not fully utilized.

Operations on Arrays

Arrays support several fundamental operations that allow the user to access, modify, and organize data efficiently.
The most common operations performed on arrays are **Traversal, Insertion, Deletion, Searching,** and **Sorting**.

Basic Operations on Arrays

- Traversal – Accessing each element of the array.
- Insertion – Adding an element at a specific position.
- Deletion – Removing an element.
- Searching – Finding an element by value or index.
- Sorting – Arranging elements in a specific order

1. Traversal

Definition:
Traversal means accessing and processing each element of the array exactly once, usually through a loop.

Example (C):

```
for(int i = 0; i < n; i++) {
   printf("%d ", arr[i]);
}
```

Explanation:
The loop runs from index 0 to n-1 to display or process all the elements of the array.

2. Insertion

Definition:
Insertion is the process of adding a new element into the array at a specified position.

Steps:

1. Shift all elements to the right from the desired position.
2. Place the new element at that position.
3. Increase the array size.

Example (C):

```
for(int i = n; i > pos; i--) {
   arr[i] = arr[i-1];
}
arr[pos] = newElement;
n++;
```

Note:
Insertion takes **O(n)** time since elements may need to be shifted.

3. Deletion

Definition:
Deletion removes an element from a specific position in the array.

Steps:

1. Find the position of the element to be deleted.
2. Shift all the elements after that position one step to the left.
3. Reduce the array size by one.

Example (C):

```
for(int i = pos; i < n-1; i++) {
   arr[i] = arr[i+1];
```

}

n--;

Note:
Like insertion, deletion also takes **O(n)** time due to shifting elements.

4. Searching

Definition:
Searching is the process of finding the location of a given element in the array.

Types:

- **Linear Search:** Checks each element one by one.
- for(int i = 0; i < n; i++) {
- if(arr[i] == key)
- return i;
- }
- **Binary Search:** Works on sorted arrays; divides the search range in half repeatedly.
- while(low <= high) {
- mid = (low + high)/2;
- if(arr[mid] == key)
- return mid;
- else if(arr[mid] < key)
- low = mid + 1;
- else
- high = mid - 1;
- }

Time Complexity:

- Linear Search → O(n)
- Binary Search → O(log n)

---

5. Sorting

Definition:
Sorting arranges the elements of an array in ascending or descending order.

Common Algorithms:

- **Bubble Sort:** Repeatedly compares adjacent elements and swaps them if out of order.
- **Selection Sort:** Finds the smallest element and places it in the correct position.
- **Insertion Sort:** Builds the sorted array one element at a time.

Example (Bubble Sort in C):

```
for(int i = 0; i < n-1; i++) {
   for(int j = 0; j < n-i-1; j++) {
      if(arr[j] > arr[j+1]) {
         int temp = arr[j];
         arr[j] = arr[j+1];
         arr[j+1] = temp;
      }
   }
}
```

Result:
Array elements are arranged in sorted order.

Multiple Choice Questions

Multiple Choice Questions on Arrays

1. What is an array?
A) A data structure that stores elements in a non-contiguous memory
B) A collection of elements of different data types
C) A collection of elements of the same data type stored in contiguous memory locations
D) None of the above
Answer: C

2. What is the index of the first element in an array?
A) 1
B) 0
C) -1
D) Depends on the compiler
Answer: B

3. Which of the following operations is used to access each element of an array?
A) Searching
B) Traversal
C) Sorting
D) Deletion
Answer: B

4. Which operation involves adding an element at a specific position in an array?
A) Deletion
B) Traversal
C) Insertion
D) Sorting
Answer: C

5. When an element is deleted from an array, what happens to the remaining elements?

A) They remain in the same position
B) They shift to fill the empty space
C) They are removed as well
D) Array size increases
Answer: B

6. The process of rearranging elements of an array in ascending or descending order is called:
A) Searching
B) Traversal
C) Sorting
D) Insertion
Answer: C

7. In linear search, each element is compared:
A) Only once
B) Until the required element is found or the array ends
C) Only in the middle
D) Randomly
Answer: B

8. Which of the following sorting algorithms has the best average time complexity?
A) Bubble Sort
B) Insertion Sort
C) Quick Sort
D) Selection Sort
Answer: C

9. In which operation do we move sequentially through all elements of an array?
A) Traversal
B) Sorting
C) Searching
D) Deletion
Answer: A

10. What happens if you try to access an array element beyond its size limit?
A) Program gives an error
B) Program crashes or shows undefined behavior
C) Element value becomes zero
D) Compiler fixes it automatically
Answer: B

Flipped Classroom Feedback

Name of the student:

Semester:

1. How useful did you find the flipped classroom activity on Arrays?
A) Very useful
B) Somewhat useful
C) Not useful
D) Didn't participate

2. How easy was it to understand the topic after the flipped classroom activity?
A) Very easy
B) Easy
C) Difficult
D) Very difficult

3. Did pre-class preparation help you understand the topic better in class?
A) Yes, a lot
B) Somewhat
C) No, not really
D) Not at all

4. How engaging was the in-class discussion?
A) Very engaging
B) Moderately engaging
C) Less engaging
D) Not engaging

5. Did the flipped classroom help you improve your problem-solving skills?
A) Yes, significantly
B) Yes, a little

C) Not much
D) Not at all

6. Did the examples and practice exercises help you understand the topic better?
A) Strongly agree
B) Agree
C) Disagree
D) Strongly disagree

7. How likely are you to participate in another flipped classroom activity?
A) Very likely
B) Likely
C) Unlikely
D) Very unlikely

8. Did the flipped classroom save time in understanding the topic?
A) Yes, a lot
B) Yes, a little
C) Not really
D) Not at all

9. How clear were the instructions for pre-class preparation?
A) Very clear
B) Clear
C) Unclear
D) Very unclear

10. Overall, how would you rate the flipped classroom experience?
A) Excellent
B) Good
C) Average
D) Poor

Flipped Classroom Feedback

Name of the student:

Semester:

1. How useful did you find the flipped classroom activity on Arrays?
A) Very useful
B) Somewhat useful
C) Not useful
D) Didn't participate

2. How easy was it to understand the topic after the flipped classroom activity?
A) Very easy
B) Easy
C) Difficult
D) Very difficult

3. Did pre-class preparation help you understand the topic better in class?
A) Yes, a lot
B) Somewhat
C) No, not really
D) Not at all

4. How engaging was the in-class discussion?
A) Very engaging
B) Moderately engaging
C) Less engaging
D) Not engaging

5. Did the flipped classroom help you improve your problem-solving skills?
A) Yes, significantly
B) Yes, a little
C) Not much
D) Not at all

6. Did the examples and practice exercises help you understand the topic better?
A) Strongly agree
B) Agree
C) Disagree
D) Strongly disagree

7. How likely are you to participate in another flipped classroom activity?
A) Very likely
B) Likely
C) Unlikely
D) Very unlikely

8. Did the flipped classroom save time in understanding the topic?
A) Yes, a lot
B) Yes, a little
C) Not really
D) Not at all

9. How clear were the instructions for pre-class preparation?
A) Very clear
B) Clear
C) Unclear
D) Very unclear

10. Overall, how would you rate the flipped classroom experience?
A) Excellent
B) Good
C) Average
D) Poor